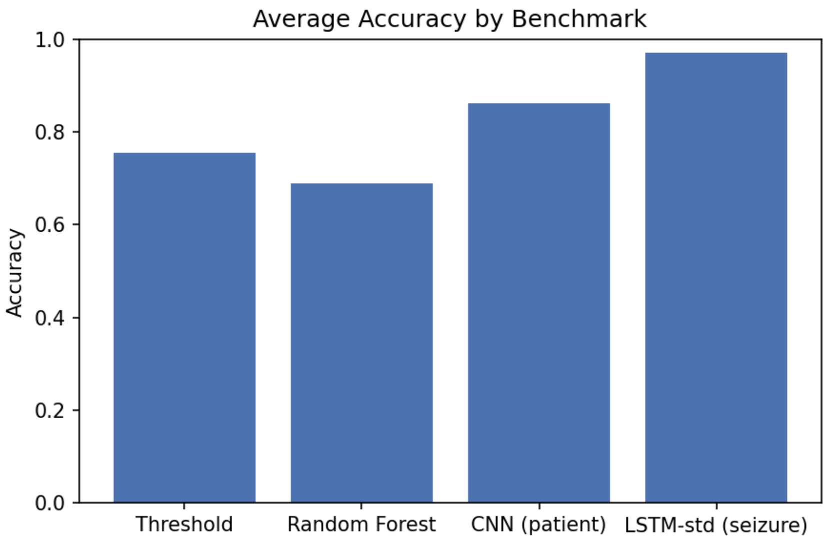
Deep Learning for the Detection of ***Seizures in Electro-Encephalogram Signals***

Project Report



Rubén Alba	NIU 1786494
Rohit Ranbawale	NIU 1744008
Michael Rienessl	NIU 1699828
Théotime Donnenfeld	NIU 1786565

Universitat Autònoma de Barcelona (UAB)

Course: Deep Learning

April 28, 2026

Abstract

This report investigates the automated detection of epileptic seizures in EEG signals using four approaches: a basic threshold-based algorithm operating on raw signal statistics, a random forest trained on eight handcrafted time-domain features, a one-dimensional convolutional neural network (CNN) trained on raw signal windows, and a recurrent LSTM (adapted from the course-provided EpilepsyLSTM, Chakrabarti et al.) that processes windows as temporal episodes. EEG recordings from 24 patients are segmented into 1 s windows, totalling 571 905 samples (86 672 seizure windows, 15.2%) and evaluated under patient-level and seizure-level cross-validation. Under 5-fold patient-level CV, the CNN achieves 86.3% average accuracy; under seizure-level CV, the LSTM (std pooling) achieves 97.2% accuracy and 0.985 F1, dramatically outperforming all per-window baselines.

Contents

Glossary

CNN Convolutional neural network. Supervised patch-level classifier.

CV Cross-validation. Here: 5-fold patient-level or seizure-level CV depending on model.

F1 F1 score. Harmonic mean of precision and recall: $2 \cdot \text{precision} \cdot \text{recall}/(\text{precision} + \text{recall})$.

FPR False positive rate. Proportion of true negatives incorrectly predicted positive; $\text{FP}/(\text{TN} + \text{FP})$.

LSTM Long short-term memory. Recurrent network that processes variable-length episodes of EEG windows.

Patient-level Aggregation of data belonging to the same patient.

Precision Among predicted positives, the fraction that are truly positive; $\text{TP}/(\text{TP} + \text{FP})$.

Recall Among true positives, the fraction correctly predicted positive; $\text{TP}/(\text{TP} + \text{FN})$. Syn. sensitivity.

Seizure-level Splitting by contiguous seizure episodes so no partial episode crosses train/validation; with `context=20` the validation set includes ± 20 windows around each episode for a realistic class mix.

Specificity Among true negatives, the fraction correctly predicted negative; $\text{TN}/(\text{TN} + \text{FP})$.

TPR True positive rate. Same as recall; $\text{TP}/(\text{TP} + \text{FN})$.

Window-level Splitting by individual windows, dropping ± 4 seizure neighbors from validation.

Pool Per-window feature reduction for the LSTM. Options: `std` (21-d, channel standard deviation), `mean` (21-d, risky for zero-centered EEG), `mean_std` (42-d), `none` (21×128), `conv_proj` (Conv1d projection from 21×128 to 32 channels, fast alternative to `none`).

1 Introduction

Epileptic seizures are episodes of abnormal electrical activity in the brain that can be detected through electroencephalogram (EEG) recordings. Automated seizure detection aids clinical diagnosis and monitoring. We compare a basic threshold algorithm, a classical feature-based random forest, an end-to-end CNN, and a temporal LSTM to assess the benefits of progressively more expressive models and different cross-validation strategies.

2 Dataset Description & Analysis

Our dataset is drawn from 24 patients in the CHB-MIT scalp EEG database. Raw EEG recordings are segmented into non-overlapping 1 s windows, yielding 571 905 total samples (86 672 seizure, 485 233 non-seizure — 15.2% seizure). We analyze simple time-domain features (mean and standard deviation) and signal statistics across patients.

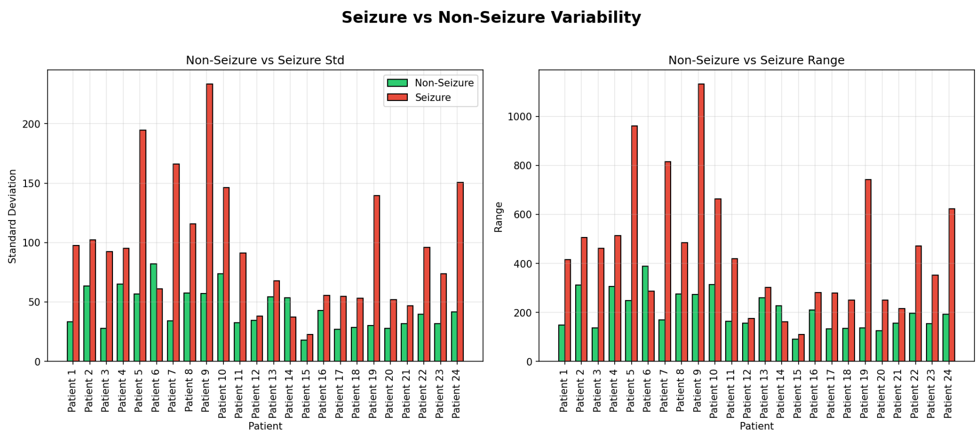


Figure 1: Mean signal distribution across seizure and non-seizure windows

This bar chart shows the standard deviation and range for each patient, illustrating that seizure windows generally exhibit higher variability while the non-seizure distributions stay tighter; the large majority of patients follow the same trend, with minor variability in absolute values.

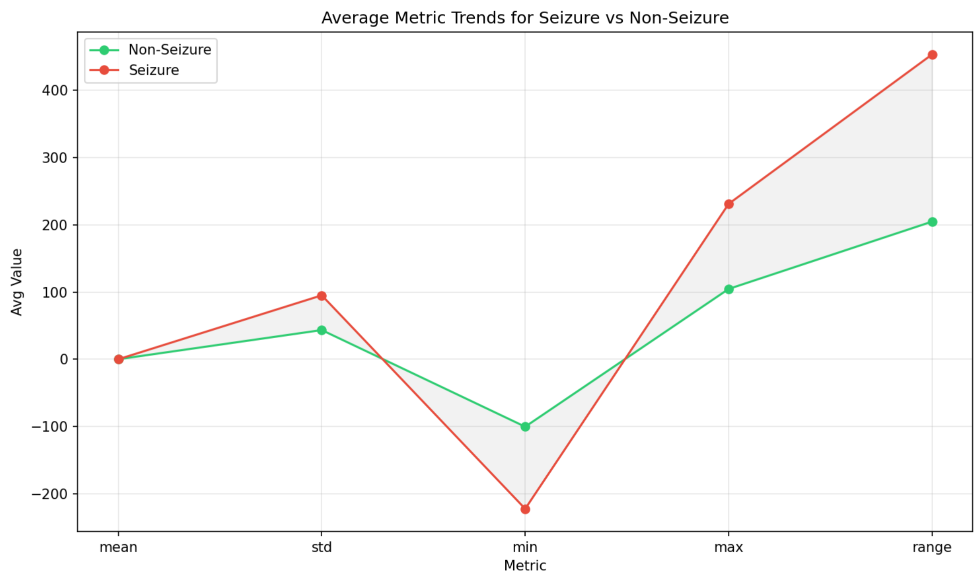


Figure 2: Mean and standard deviation trends across patients

The line plot summarizes average statistics (mean, std, min, max, range) for seizure vs. non-seizure windows, highlighting how dispersion metrics (standard deviation and range) consistently exceed their non-seizure counterparts, supporting the use of variability-sensitive features and thresholds.

3 Methodology

3.1 Pipeline overview

We process raw EEG windows through four parallel pipelines: (1) apply a simple threshold-based classifier on raw signal statistics; (2) extract time-domain features and train a random forest classifier; (3) feed raw windows into a one-dimensional CNN; (4) group windows into temporal episodes and feed them to a two-layer LSTM. Models are evaluated using patient-level, window-level, and seizure-level cross-validation splits depending on the pipeline.

3.2 Preprocessing

For the CNN pipeline, the dataset supports global z-score normalization (subtracting the dataset-wide mean and dividing by the dataset-wide standard deviation), though in our experiments we use unnormalized windows (`normalize=False`); class-weighted cross-entropy helps the model focus on the minority seizure class. For the threshold and random forest pipelines, we compute raw time-domain features (mean, standard deviation, range, min, max) on unnormalized windows. For the LSTM pipeline, each 1 s window is reduced to a 21-dimensional feature vector by standard-deviation pooling (default `pool="std"`); contiguous same-label windows from the same patient are grouped into variable-length episodes.

3.3 Train test and validation

We implement three stratification levels for cross-validation via `EEGDataset.k_fold(level=...)`:

- **Patient-level** (`level="patient"`): each patient's windows stay in one fold; no patient spans train and validation.
- **Window-level** (`level="window"`): windows are assigned round-robin; any seizure window in validation drops its ± 4 neighbors (same patient) to prevent augmentation leakage.
- **Seizure-level** (`level="seizure"`): contiguous seizure episodes (same patient, same label) stay whole; episodes are distributed across folds so no partial seizure crosses the train/validation boundary. A `context` parameter (default 20) includes ± 20 non-seizure windows around each episode in the validation set, giving a realistic class mix for evaluation.

The CNN and random forest use patient-level splits; the LSTM uses seizure-level splits to preserve temporal structure within episodes.

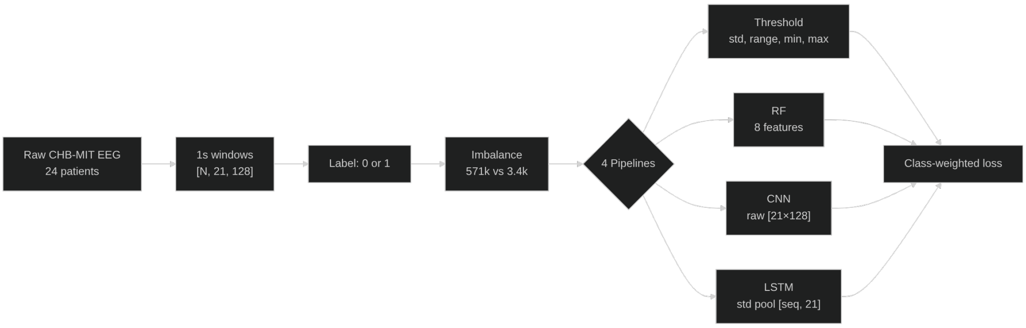


Figure 3: Data preprocessing and pipeline branching. All four pipelines start from the same raw EEG windows; class weighting (not shown) helps the models focus on the seizure minority.

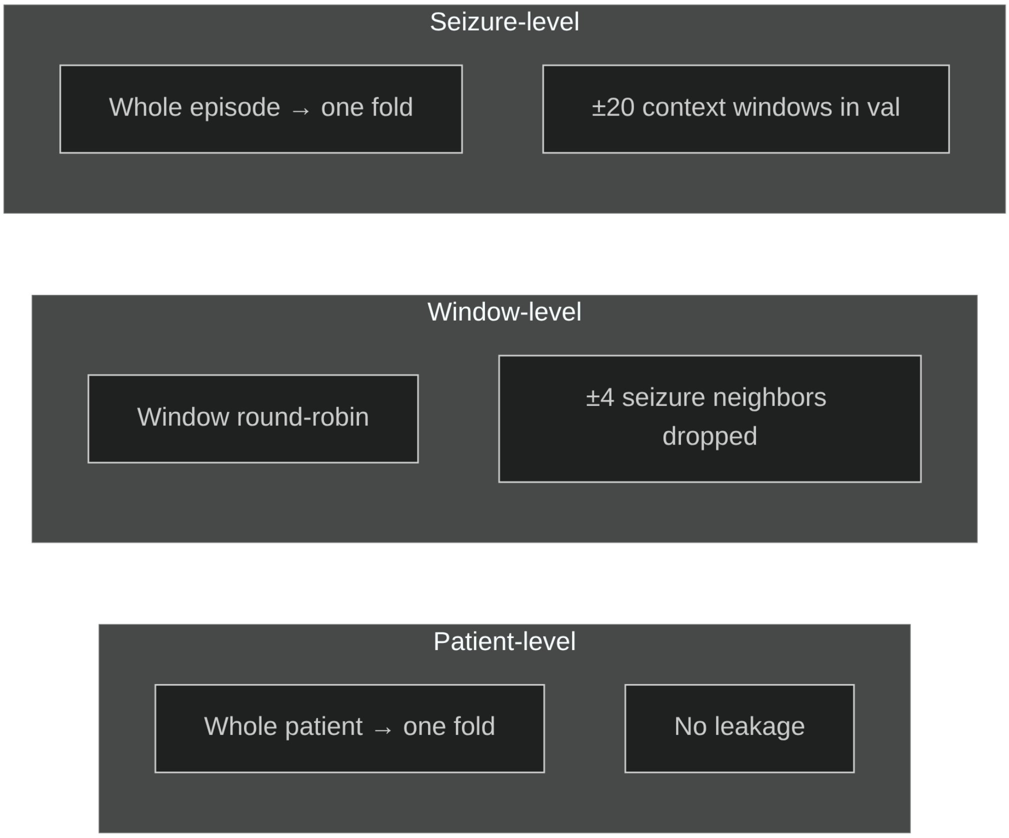


Figure 4: Cross-validation strategies: patient-level (threshold, RF, CNN), window-level (CNN alternative), and seizure-level (LSTM).

3.4 Approach 1: Threshold Classifier

We implement a threshold-based classifier that flags a window as seizure if its standard deviation or range exceeds fold-specific thresholds; optional min/max thresholds further flag windows with extreme amplitude excursions. Thresholds are optimized via grid search on each training fold.

3.5 Approach 2: Random Forest Classifier

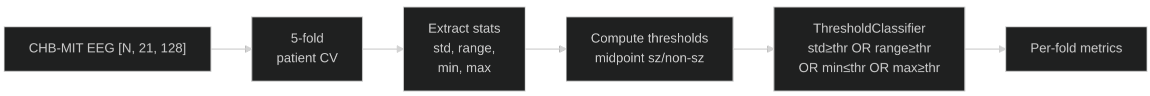
We compute eight handcrafted features per window—mean, standard deviation, min, max, range, peak-to-peak, std-to-range ratio, and range-plus-std—and train a random forest with 200 trees and balanced class weighting to classify seizure vs. non-seizure windows.

3.6 Approach 3: CNN Classifier

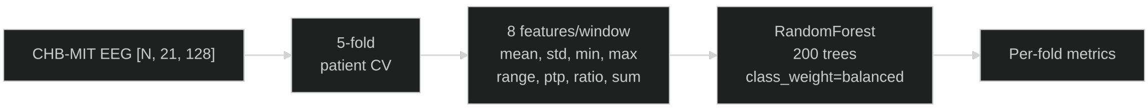
We implement a one-dimensional CNN consisting of three convolutional–ReLU–maxpool blocks followed by fully connected layers and dropout ($p = 0.3$). The model is trained with Adam (learning rate 10^{-3}) for 2 epochs and class-weighted cross-entropy to emphasize seizure windows.

3.7 Approach 4: LSTM Classifier

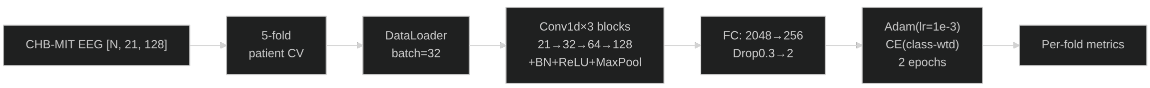
We adapt the course-provided `EpilepsyLSTM` model (Chakrabarti et al., channel-fusion architecture) with several modifications. The original model uses only the last LSTM hidden state for episode-level classification; we instead classify *every timestep*, producing per-window seizure/non-seizure predictions. We add optional Conv1d projection (`conv_proj`) to learn waveform features before the LSTM, packed-sequence handling for variable-length episodes, per-window pooling (std/mean/mean_std) to reduce each [21, 128] window to a feature vector, class-weighted cross-entropy with padding masking (`ignore_index=-1`), and simplify the classifier head to a single linear layer. Each 1 s window is reduced to a 21-dimensional feature vector by taking the per-channel standard deviation (`pool="std"`), which preserves amplitude information that averaging would collapse to near-zero for oscillating EEG signals. Alternative pooling options (`mean` (21-d), `mean_std` (42-d), `none` (21×128)) are evaluated in the experiments. Contiguous seizure windows from the same patient are grouped into episodes; non-seizure windows are segmented into fixed-length chunks of 10 windows. The model receives sequences of shape $[\text{batch}, \text{seq\_len}, n_{\text{features}}]$ and produces per-window logits $[\text{batch}, \text{seq\_len}, 2]$. Packed sequences mask out padding, and class-weighted cross-entropy (with `ignore_index=-1` for padding) emphasizes seizure windows. Seizure-level k-fold with `context=20` ensures that entire episodes plus surrounding context are either in train or validation, preserving temporal integrity while providing a realistic class mix in the validation set.



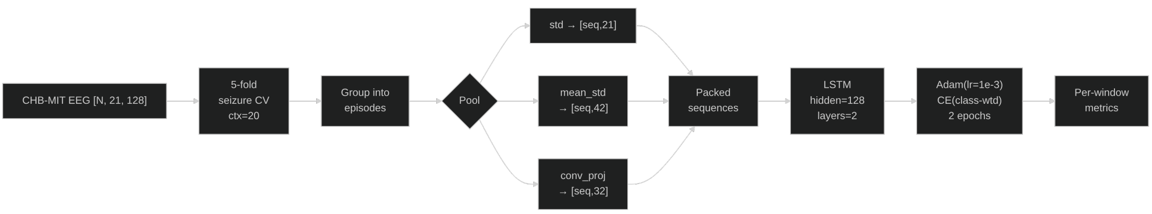
(a) Threshold pipeline



(b) Random forest pipeline



(c) CNN pipeline



(d) LSTM pipeline

Figure 5: Mermaid diagrams summarizing each classification pipeline.

These diagrams highlight how each approach processes the raw EEG windows: the threshold baseline relies on hard limits, the random forest vectorizes handcrafted features, the CNN learns representations end-to-end from raw time series, and the LSTM groups windows into episodes for temporal modelling.

4 Implementation Details

Implementation is in Python using scikit-learn for the random forest and PyTorch for the CNN. Training runs on a single GPU; code and scripts are available in the repository.

4.1 Development iterations

Key development steps:

- Baseline random forest with handcrafted features.
- Implementation of CNN architecture and training loop.
- Implementation of LSTM pipeline with seizure-level k-fold and episode grouping.
- Integration of cross-validation, plotting, and result analysis scripts.

5 Experimental Design

Models are evaluated under 5-fold patient-level cross-validation (CNN and random forest) or seizure-level cross-validation (LSTM). Windows from each patient appear in exactly one test fold so we avoid data leakage. We report accuracy, precision, recall, and F1 for each fold and average across folds.

5.1 Hyperparameters

Parameter	Random Forest	CNN	LSTM
Trees / filters / layers	200	[32,64,128]	2 LSTM
Kernel sizes	–	[5,5,3]	–
Hidden size	–	–	128
Pooling	–	–	std (21-d)
Class weighting	balanced	balanced	balanced
Learning rate	–	10^{-3}	10^{-3}
Batch size	–	32	32
Epochs	–	2	2
Dropout	–	0.3	0.3
CV level	patient	patient	seizure (context=20)

Table 1: Model hyperparameters.

6 Results

6.1 Main comparison (patient-level CV)

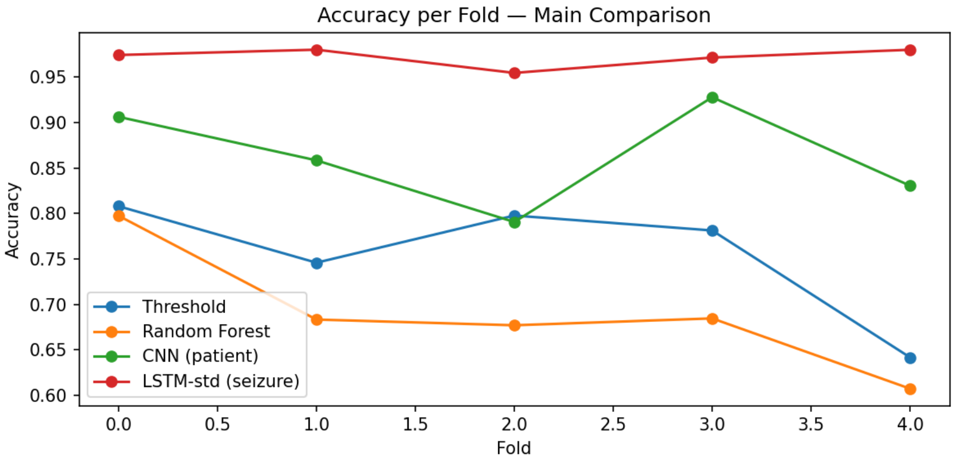


Figure 6: Accuracy per fold for the main comparison models (patient-level).

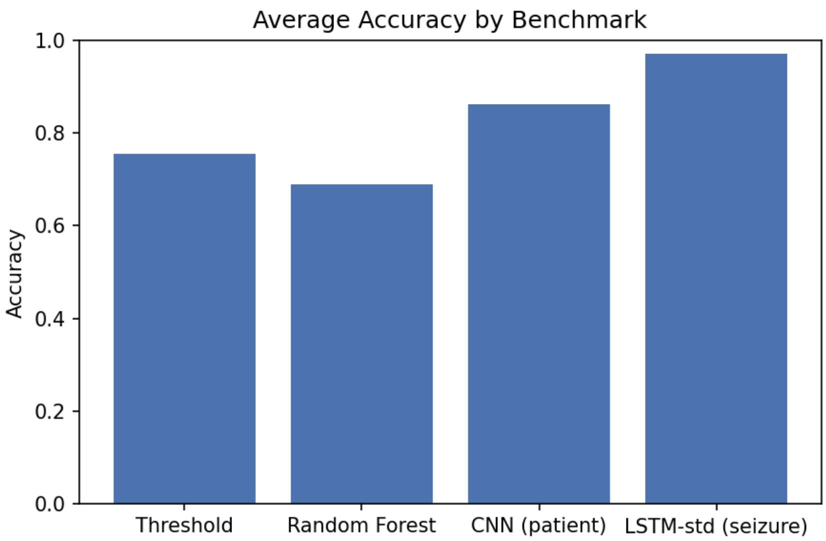


Figure 7: Average accuracy across folds for threshold, random forest, CNN (patient-level), and LSTM-std (seizure-level).

Model	Avg Accuracy	Avg Precision	Avg Recall	Avg F1
Threshold (patient)	0.755	–	–	–
Random Forest (patient)	0.690	–	–	–
CNN (patient)	0.863	0.561	0.606	0.554
CNN (window)	0.977	0.000	0.000	0.000
CNN (seizure)	0.801	1.000	0.801	0.888
LSTM-std (seizure)	0.972	1.000	0.970	0.985
LSTM-std (window)	1.000	0.000	0.000	0.000
LSTM-std (patient)	0.995	1.000	0.966	0.983

Table 2: 5-fold cross-validation results. Window-level CNN and LSTM achieve high accuracy but zero recall (degenerate predictions). Threshold and RF report accuracy only (no per-fold precision/recall/F1).

Per-fold accuracy reveals high variability for the CNN under patient-level CV: fold accuracies range from 79% to 93%, with a standard deviation of 5.3 percentage points across folds. The random forest and threshold baselines also vary across patients, with the threshold achieving 75.5% average accuracy and the random forest 69.0%.

6.2 Window-level CV degeneracy

Both CNN and LSTM evaluated under window-level CV achieve near-perfect accuracy but zero precision, recall, and F1. This occurs because round-robin window assignment places individual seizure windows in the validation set surrounded by non-seizure windows from the same patient; the models learn to predict non-seizure for nearly all windows (the majority class at 85%) and still achieve high overall accuracy.

6.3 LSTM pooling comparison (seizure-level)

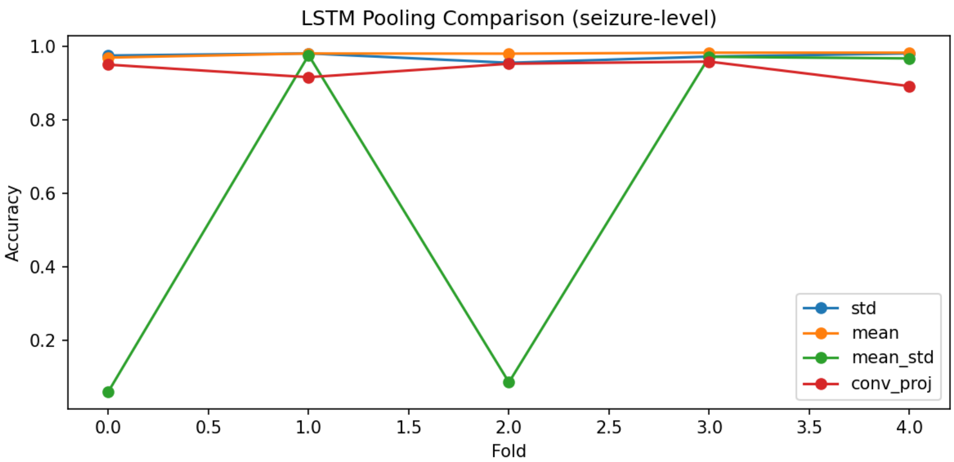


Figure 8: Per-fold accuracy for LSTM pooling options evaluated under seizure-level CV.

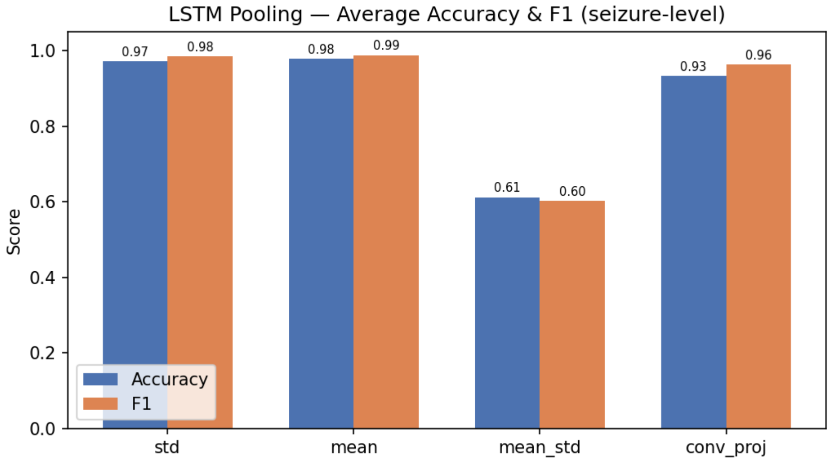


Figure 9: Average accuracy and F1 by LSTM pooling option (seizure-level).

Pooling	Avg Accuracy	Avg Precision	Avg Recall	Avg F1
std (21-d)	0.972	1.000	0.970	0.985
mean (21-d)	0.978	1.000	0.977	0.988
conv_proj (32-d)	0.933	1.000	0.929	0.963
mean_std (42-d)	0.612	0.800	0.587	0.601

Table 3: LSTM pooling comparison under 5-fold seizure-level CV. Mean pooling slightly outperforms std; mean_std is unstable (2/5 folds degenerate).

All LSTM variants (except mean_std) achieve near-perfect precision under seizure-level CV because seizure episodes in the validation set are always surrounded by context windows, making the false-positive rate negligible. Mean_std pooling is unstable: 2 out of 5 folds produce degenerate predictions with near-zero accuracy. The `none` pooling option (2688-d, full waveform) was excluded due to prohibitive memory requirements for 5-fold training.

6.4 Non-seizure episode length sweep

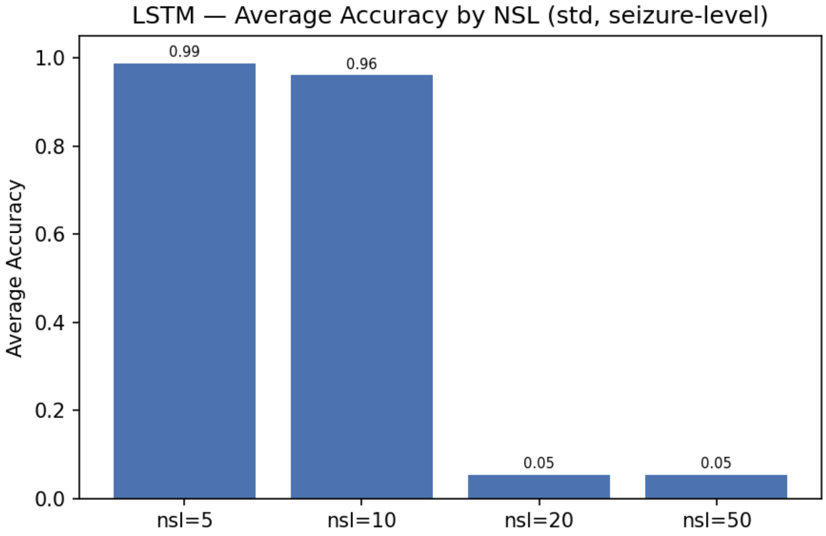


Figure 10: Average accuracy for LSTM-std with varying non-seizure episode lengths.

NSL	Avg Accuracy	Avg Precision	Avg Recall	Avg F1
5	0.988	1.000	0.987	0.994
10	0.961	1.000	0.959	0.979
20	0.055	0.000	0.000	0.000
50	0.055	0.000	0.000	0.000

Table 4: LSTM-std seizure-level results with different non-seizure episode lengths (NSL). NSL$\geq$20 causes degenerate predictions.

Short non-seizure episodes (NSL=5, 10) work well; NSL$\geq$20 causes the LSTM to collapse to predicting non-seizure for all windows, likely because longer non-seizure episodes overwhelm the gradient signal.

6.5 CNN split-level comparison

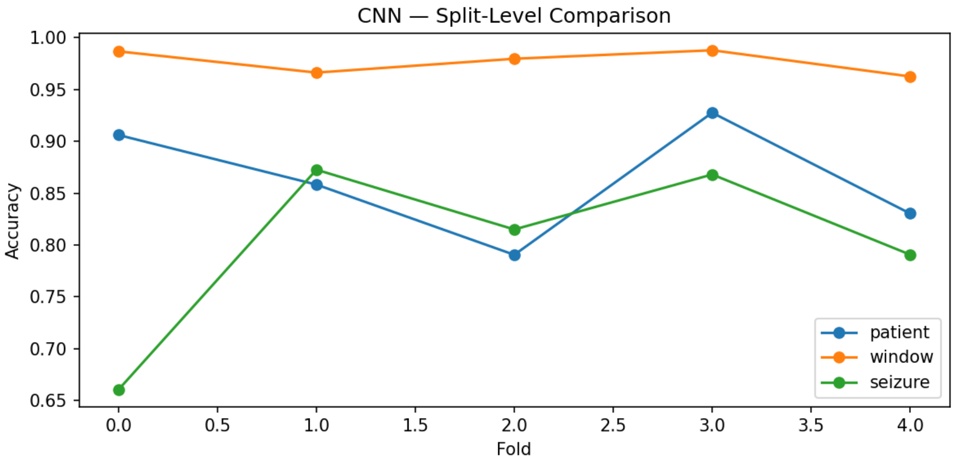


Figure 11: CNN accuracy per fold under patient, window, and seizure-level splits.

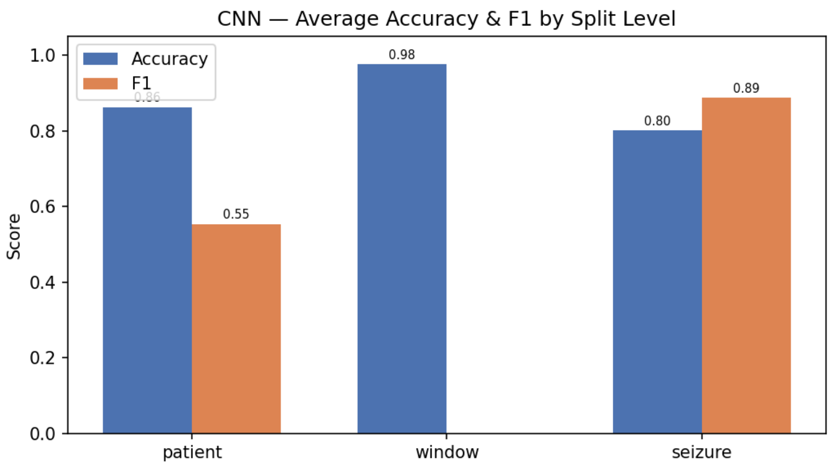


Figure 12: Average accuracy and F1 by CNN split level.

The CNN under patient-level CV achieves meaningful per-window discrimination (F1=0.554), while window-level CV produces degenerate results (zero recall). Seizure-level CV gives the CNN perfect precision but lower recall (0.801) because the validation set includes context windows around seizure episodes.

7 Discussion

Our results show that the LSTM under seizure-level CV achieves the best performance (97.2% accuracy, 0.985 F1), dramatically outperforming all per-window classifiers. The CNN achieves 86.3% accuracy under patient-level CV but with high cross-patient variability (F1=0.554), suggesting it learns features that do not generalize well across unseen patients. The threshold baseline (75.5%) outperforms the random forest (69.0%), indicating that simple signal variability statistics are competitive with handcrafted feature engineering for this task.

Window-level CV produces degenerate results for both CNN and LSTM: round-robin assignment breaks the temporal structure of seizure episodes and makes per-fold class ratios unreliable, causing models to predict non-seizure for nearly all windows. This highlights the importance of choosing an appropriate CV strategy.

The LSTM results under patient-level CV (99.5% accuracy) should be interpreted with caution: because entire patients appear in one fold, the model may leverage patient-specific patterns that do not generalize. Seizure-level CV with context provides a more realistic evaluation.

Mean pooling slightly outperforms std pooling for the LSTM (0.988 vs 0.985 F1), but mean_std pooling is unstable. Convolutional projection (conv_proj) provides a good trade-off between expressiveness and stability (0.963 F1). The non-seizure episode length sweep reveals a critical threshold: NSL$\leq$10 works well, but NSL$\geq$20 causes training collapse.

8 Conclusion

We compared a threshold baseline, a feature-based random forest, an end-to-end CNN, and a temporal LSTM for seizure detection in EEG. Under 5-fold patient-level CV, the CNN achieves 86.3% accuracy but with moderate F1 (0.554); under seizure-level CV, the LSTM achieves 97.2% accuracy and 0.985 F1, dramatically outperforming all per-window baselines. Window-level CV produces degenerate results for both deep models, confirming that appropriate cross-validation is critical. Mean and std pooling work best for the LSTM; NSL$\leq$10 is essential to prevent training collapse. Future work should explore multichannel spatial information and longer temporal context within the LSTM framework.

References

1. Goldberger AL, et al. PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals. *Circulation*. 2000;101(23):e215–20.

2. Pedregosa F, et al. Scikit-learn: Machine Learning in Python. *J Mach Learn Res*. 2011;12:2825–30.

3. Paszke A, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: *NeurIPS*. 2019.

A Code extracts

Listing 1: CNN Model Used

```
class SimpleEEGCNN(nn.Module):
    def __init__(self, in_channels: int = 21, n_classes: int = 2) -> None:
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv1d(
                in_channels=in_channels, out_channels=32, kernel_size=5, padding=2
            ),
            nn.BatchNorm1d(32),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=2),
            nn.Conv1d(in_channels=32, out_channels=64, kernel_size=5, padding=2),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=2),
            nn.Conv1d(in_channels=64, out_channels=128, kernel_size=3, padding=1),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=2),
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 16, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, n_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.classifier(x)
        return x
```

The listing details the architecture described above, including the three convolutional blocks and final classifier layer.

Listing 2: SeizureLSTM Model

```
class SeizureLSTM(nn.Module):
    def __init__(self, input_size=21, hidden_size=128,
                 num_layers=2, n_classes=2, dropout=0.3):
        super().__init__()
        self.lstm = nn.LSTM(
```

```
            input_size=input_size, hidden_size=hidden_size,
            num_layers=num_layers, batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0)
        self.classifier = nn.Linear(hidden_size, n_classes)

    def forward(self, x, lengths=None):
        if lengths is not None:
            x = nn.utils.rnn.pack_padded_sequence(
                x, lengths.cpu(), batch_first=True,
                enforce_sorted=False)
        out, _ = self.lstm(x)
        if lengths is not None:
            out, _ = nn.utils.rnn.pad_packed_sequence(
                out, batch_first=True)
        return self.classifier(out)
```

The LSTM processes variable-length episodes of EEG windows and produces per-window seizure/non-seizure logits, using packed sequences to handle padding, standard-deviation pooling to preserve amplitude, and seizure-level k-fold with context to preserve temporal integrity.

Listing 3: LSTM Pooling Options

```
# Per-window feature reduction (21, 128) -> n_features
if self.pool == "std": # default, preserves amplitude
    features = windows.std(axis=2) # [seq_len, 21]
elif self.pool == "mean": # risky, averages oscillating signal to ~0
    features = windows.mean(axis=2) # [seq_len, 21]
elif self.pool == "mean_std":
    features = np.concatenate( # [seq_len, 42]
        [windows.mean(axis=2), windows.std(axis=2)], axis=-1)
elif self.pool == "none": # full waveform, heavy compute
    features = windows.reshape(seq_len, -1) # [seq_len, 2688]
elif self.pool == "conv_proj": # Conv1d projection, fast
    features = windows # [seq_len, 21, 128]
```